

# torch.fft.hfft#

<https://pytorch.org/docs/stable/generated/torch.fft.hfft.html>

## Description:

Computes the one dimensional discrete Fourier transform of a Hermitian symmetric input signal. `hfft()/ihfft()` are analogous to `rfft()/irfft()`. The real FFT expects a real signal in the time-domain and gives a Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the length argument `n`, in the same way as with `irfft()`. Because the signal is Hermitian in the time-domain, the result will be real in the frequency domain. Note that some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in `input[0]` would result in one or more complex frequency terms which cannot be represented in a real output and so will always be ignored.

## Function Signature

---

```
torch.fft.hfft(input, n=None, dim=-1, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

## Parameters

---

### Keyword

out (Tensor, optional) – the output tensor.

## Code Examples

---

```
>>> t = torch.linspace(0, 1, 5)
>>> t
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
>>> T = torch.fft.ifft(t)
>>> T
tensor([ 0.5000-0.0000j, -0.1250-0.1720j, -0.1250-0.0406j,
        -0.1250+0.0406j,
         -0.1250+0.1720j])
```

```
>>> torch.fft.hfft(T[:3], n=5)
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

```
>>> torch.fft.hfft(T[:3])
tensor([0.1250, 0.2809, 0.6250, 0.9691])
```

## Notes & Warnings

---

**NOTE**

Note `hfft()/ihfft()` are analogous to `rfft()/irfft()`. The real FFT expects a real signal in the time-domain and gives a Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the length argument `n`, in the same way as with `irfft()`.

**NOTE**

Note Because the signal is Hermitian in the time-domain, the result will be real in the frequency domain. Note that some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in `input[0]` would result in one or more complex frequency terms which cannot be represented in a real output and so will always be ignored.

**NOTE**

Note The correct interpretation of the Hermitian input depends on the length of the original data, as given by `n`. This is because each input shape could correspond to either an odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. So, it is recommended to always pass the signal length `n`.

**NOTE** Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. With default arguments, size of the transformed dimension should be  $(2^n + 1)$  as argument  $n$  defaults to even  
output size =  $2 * (\text{transformed\_dim\_size} - 1)$

## Detailed Documentation

---

### Documentation

Computes the one dimensional discrete Fourier transform of a Hermitian symmetric input signal.

hfft()/ihfft() are analogous to rfft()/irfft(). The real FFT expects a real signal in the time-domain and gives a Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the length argument  $n$ , in the same way as with irfft().

Because the signal is Hermitian in the time-domain, the result will be real in the frequency domain. Note that some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in input[0] would result in one or more complex frequency terms which cannot be represented in a real output and so will always be ignored.

The correct interpretation of the Hermitian input depends on the length of the original data, as given by  $n$ . This is because each input shape could correspond to either an

odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. So, it is recommended to always pass the signal length  $n$ .

Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. With default arguments, size of the transformed dimension should be  $(2^n + 1)$  as argument  $n$  defaults to even output size =  $2 * (\text{transformed\_dim\_size} - 1)$

`input (Tensor)` – the input tensor representing a half-Hermitian signal

`n (int, optional)` – Output signal length. This determines the length of the real output. If given, the input will either be zero-padded or trimmed to this length before computing the Hermitian FFT. Defaults to even output:  $n=2*(\text{input.size(dim)} - 1)$ .

`dim (int, optional)` – The dimension along which to take the one dimensional Hermitian FFT.

`norm (str, optional)` –

Normalization mode. For the forward transform (`hfft()`), these correspond to:

"forward" - normalize by  $1/n$

"backward" - no normalization

"ortho" - normalize by  $1/\sqrt{n}$  (making the Hermitian FFT orthonormal)

Calling the backward transform (`ihfft()`) with the same normalization mode will apply an overall normalization of  $1/n$  between the two transforms. This is required to make `ihfft()` the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.

Taking a real-valued frequency signal and bringing it into the time domain gives Hermitian symmetric output:

Note that  $T[1] == T[-1].conj()$  and  $T[2] == T[-2].conj()$  is redundant. We can thus compute the forward transform without considering negative frequencies:

Like with `irfft()`, the output length must be given in order to recover an even length output: